



GitHub Trending Top 10

每日热门开源项目深度解读

2026-10-07

今日榜单

1	morluto/rea	TypeScript	Reverse engineer anything with agents, from app behavior down to native binaries...	2天
2	mattpocock/skills	Shell	Skills for Real Engineers. Straight from my .agents directory.	2天
3	boykopovar/AnyPS5	C++	Tool for automatic PS5 executables porting to Linux and Windows	3天
4	ayghri/i-have-adhd	Python	A skill to stop your coding agent from burying the answer. ADHD-friendly output.	2天
5	cathrynlavery/diagram-design	HTML	Editorial diagram design for Claude Code, Codex, GitHub Copilot, Factory Droid, ...	NEW
6	addyosmani/agent-skills	JavaScript	Production-grade engineering skills for AI coding agents.	NEW
7	EpicGames/raddebugger	C	A native, user-mode, multi-process, graphical debugger.	NEW
8	thedotmack/claude-mem	TypeScript	Persistent Context Across Sessions for Every Agent – Captures everything your ag...	3天
9	manaflow-ai/cmux	Swift	Open source Ghostty-based macOS terminal with vertical tabs and notifications fo...	NEW
10	trycua/cua	Rust	Scale computer-use 2.0 with open-source drivers, cross-OS fleets, and benchmarks...	NEW

#1

TypeScript

连续 2 天

rea

Reverse engineer anything with agents, from app behavior down to native binaries.

Stars **13,185** Forks **1,388** Today **+4,666**

<https://github.com/morluto/rea>

项目简介：REA 是一个基于 MCP (Model Context Protocol) 协议的反向工程工具集，目标是让 AI Agent 能够"理解任何东西的实现原理"——从应用功能行为一直追溯到原生二进制层面。它解决的核心问题是：当你看到一个应用里喜欢的功能，想搞清楚它怎么实现的，过去需要人工使用 IDA、Hopper 等工具逐步分析，现在可以交给 AI Agent 自动完成调查并给出证据链。

核心功能：一是跨层级的逆向分析能力，覆盖原生二进制、JavaScript/Electron 应用、.NET 程序集和网站；二是通过 MCP 把分析工具接入各类 AI 编程助手（Claude Code、Cursor、Codex 等），让 Agent 自主调用工具链完成调查；三是本地化分析并输出带证据和局限性的结论，而非黑箱式答案；四是支持接入已有的 Hopper 或 Ghidra 安装，静态 JS 分析则无需额外引擎；五是提供终端命令行方式直接使用同一套工具。

适用场景：主要面向想借鉴竞品功能实现的产品开发者、做安全审计和恶意样本分析的安全研究人员，以及学习二进制/逆向技术的工程师。典型场景是"看到某个 App 有个好功能，想让 Agent 帮我分析它是怎么做的，并在我自己的项目里复刻一版"。

技术亮点：用 TypeScript 编写并以 npm 包 (rea-agents) 分发，通过 npx 一键安装降低了使用门槛；用 MCP 协议标准化了 Agent 与逆向工具的交互，使其能适配主流 AI 助手生态；强调"证据+局限性"的输出方式，在 AI 易产生幻觉的背景下显得务实；同时支持 Hopper/Ghidra 这类专业工具作为后端，兼顾了深度与易用性。

上手建议：最直接的入口是运行 `npx rea-agents setup`，按提示选择要接入的 Agent 并审查配置改动（setup 会备份原有配置）。若只想先体验，可从终端对某个 Electron/JS 应用树跑一次分析看结果。想深入贡献的话，建议先读官网的 Guides 和 Investigation Model 文档，理解其调查工作流与工具目录设计，再从工具适配或新分析目标入手。

#2

Shell

连续 2 天

skills

Skills for Real Engineers. Straight from my .agents directory.

Stars **279,202** Forks **23,394** Today **+1,406**

<https://github.com/mattpocock/skills>

项目简介：这是 TypeScript 教育者 Matt Pocock 开源的一套 AI 编码代理（Agent）技能集合，直接来自他日常使用的 .agents 目录。它针对的是当前 AI 编码工具（如 Claude Code、Codex）常见的"跑偏"问题——比如代理没理解需求就动手、生成的代码难以调试——主张用小型、可组合、可改造的技能包替代 GSD、BMAD、Spec-Kit 这类"接管流程"的重型框架。

核心功能：

- **需求对齐（grilling session）**：通过 /grill-me、/grill-with-docs 等技能，让代理反过来向开发者提问，弥合"你想的"和"它做的"之间的沟通鸿沟。
- **workflow技能**：如 /trriage（结合 issue tracker 标签做分类）、setup-matt-pocock-skills（每个仓库初始化一次，配置 issue 追踪器、标签和文档路径）。
- **可组合的小技能设计**：每个技能都保持小巧、易改、可拼接，不绑定特定模型。
- **双安装路径**：Claude Code 插件（只读、订阅式更新）或 skills.sh（复制可编辑文件到项目里，自己改）。

适用场景：主要面向使用 Claude Code、Codex 等 AI 编码代理做真实工程（而非"氛围编程"）的开发者。适合那些觉得重型 AI workflow框架太"黑盒"、出问题难排查，希望保留流程控制权、并能按自己团队习惯改造技能的人。也适合想学习如何为编码代理设计 prompt/技能的人参考。

技术亮点：项目本身是 Shell 脚本，但亮点在理念——用"小而可组合"对抗"大而全"，把工程经验沉淀成可编辑的纯文本技能文件。安装器 `npx skills@latest add` 支持按需挑选技能并指定目标代理，还提供了 ADR（架构决策记录）说明为何以 Claude Code 插件形式发布，工程化思路清晰。

上手建议：最快路径是运行 `npx skills@latest add mattpocock/skills`（记得勾选 `setup-matt-pocock-skills`），然后在代理里执行一次 `/setup-matt-pocock-skills` 完成配置，接着从 `/grill-me` 体验需求对齐。想深入贡献的话，建议先读 `.agents/adr/` 下的决策记录和 `CHANGELOG.md`，理解技能的设计取舍后再动手改造或提 PR。

#3

C++

连续 3 天

AnyPS5

Tool for automatic PS5 executables porting to Linux and Windows

Stars **9,280** Forks **693** Today **+2,725**

<https://github.com/boykopovar/AnyPS5>

项目简介：AnyPS5 是一个用 C++ 编写的工具，旨在将 PS5 可执行文件自动移植到 Linux 和 Windows 平台。它通过重链接器（relinker）将可执行文件转换为目标系统的原生格式，并实现了 PS5 系统 prx 库的动态链接版本，从而在不使用模拟器或独立运行时进程的情况下运行 PS5 程序。

核心功能：第一，内置 relinker 可将 PS5 可执行文件转换为目标平台原生格式；第二，提供系统 prx 库的实现，支持动态链接；第三，包含着色器重编译器，能将 PS5 着色器转换为 SPIR-V；第四，支持 SDL 映射的游戏手柄（含模拟摇杆和扳机键）以及键鼠配置；第五，提供游戏兼容性列表和进度追踪。

适用场景：主要面向对游戏主机移植、逆向工程和系统兼容性研究感兴趣的开发者与研究者。适合希望在 PC 上运行自己合法拥有的 PS5 游戏、参与开源移植项目贡献，或研究 PS5 系统库与图形管线技术细节的人群。

技术亮点：项目采用原生格式转换而非模拟器方案，避免了传统模拟的性能开销和运行时复杂性；着色器重编译器能生成经 Spirv-Tools 验证的 SPIR-V 代码；所有不支持的状态严格抛出 `std::runtime_error` 并终止进程，便于调试和问题定位；项目还公开了技术债务文档，体现了工程透明度。

上手建议：用户可先阅读 `docs/user/USAGE.md` 了解使用方法，并查看 `docs/user/COMPATIBILITY.md` 确认自己关注的游戏是否受支持。开发者贡献前应阅读 `docs/dev/BUILD.md` 构建项目、`docs/dev/ARCHITECTURE.md` 理解架构，并遵循 `docs/dev/CONVENTIONS.md` 中的代码风格约定，同时留意 `docs/dev/TechnicalDebt.md` 中列出的已知问题。

#4

Python

连续 2 天

i-have-adhd

A skill to stop your coding agent from burying the answer. ADHD-friendly output.

Stars **54,903** Forks **3,150** Today **+620**

<https://github.com/ayghri/i-have-adhd>

项目简介：这是一个给 AI 编程助手（coding agent）用的“技能/插件”，目的是让模型输出别再绕圈子、把答案埋在长篇大论里，而是直接给结论和下一步操作。它把“ADHD 友好”的沟通原则套用到 LLM 的回复风格上，本质上是一份约束模型输出格式的提示词规范。

核心功能：一是“动作优先”，回答开头直接给下一步操作而非铺垫；二是多步骤任务强制编号，结尾只留一个具体的下一步；三是抑制跑题和寒暄，禁止“好问题！”“希望有帮助！”这类废话；四是每轮都重述当前状态，并给出以分钟计的具体时间估计；五是把列表限制在 5 项以内，让信息密度可控。

适用场景：适合日常用 Claude Code、Cursor 等编程助手写代码的开发者，尤其是被模型冗长回复反复消耗耐心的场景。当你只想快速知道“改哪一行、跑什么命令”时，这个技能能把对话从散文变成清单。

技术亮点：它本身几乎不含复杂算法，核心资产是 skills/i-have-adhd/SKILL.md 里那 10 条规则，属于典型的“提示工程即产品”。项目还提供了 fork 后自行调参的安装流程，说明它把自己定位成可定制的输出风格层，而非黑盒工具。

上手建议：直接复制 README 里那段安装指令到你的 CLI 提示中，让助手按 AGENTS.md 自行安装即可。想改风格的话，fork 仓库、编辑 SKILL.md，再按文档卸载上游版本并安装自己的副本。

#5

HTML

NEW

diagram-design

Editorial diagram design for Claude Code, Codex, GitHub Copilot, Factory Droid, and Pi. 42 diagram types. Self-contained HTML + SVG. No shadows. No Mermaid slop.

Stars **44,695** Forks **2,869** Today **+828**

<https://github.com/cathrynlavery/diagram-design>

项目简介：这是一个面向 Claude Code、Codex、GitHub Copilot 等 AI 编程助手的「技能包」，让 AI 生成的图表从千篇一律的圆角方块升级为杂志编辑级的视觉设计。它直击的痛点是：开发者让 AI 画架构图、流程图时，得到的往往是缺乏美感、与品牌风格脱节的通用图形。

核心功能：内置 42 种图表类型（架构图、流程图、时序图、ER 图、Sankey、鱼骨图、Wardley 地图等），每种提供浅色/深色/完整编辑三种静态变体；通过读取你的网站自动匹配品牌配色，60 秒完成风格对齐；支持将 draw.io、Mermaid、Excalidraw 的现有源文件按指定格式和细节层级重绘；输出为自包含的 HTML + SVG，无需构建步骤或外部依赖。

适用场景：适合需要为技术博客、产品文档、架构说明配图的开发者和技术写作者，尤其是那些不想花 30 分钟在 Figma 里调色、也不愿接受 Mermaid 默认渲染效果的人。核心使用场景是在 AI 编程助手的对话中直接调用，让 AI 生成可直接嵌入网页或文档的高质量图表。

技术亮点：最值得关注的是「语义模式与布局分离」的设计——队列、策略追踪、信任边界等行为语义可以复用最接近的现有图表类型，避免了图表类型数量爆炸。其次是完全基于静态 HTML + SVG 的实现，无 JavaScript、无外部图片依赖，任何浏览器直接打开即可渲染，同时可选配无障碍动效用于有序讲解场景。

上手建议：项目站点 diagramdesign.dev 提供了完整的图表预览和文档，建议先浏览各类型的截图确认是否符合需求。作为 AI 助手的 Skill 使用是最轻量的入口，直接在 Claude Code 等宿主中安装技能即可；若想贡献，可从新增图表类型或改进现有布局语法入手，README 中列出的 42 种类型和持续迭代的版本记录（2.0 到 2.5.10）说明扩展空间仍然很大。

#6

JavaScript

NEW

agent-skills

Production-grade engineering skills for AI coding agents.

Stars **102,556** Forks **10,749** Today **+453**

<https://github.com/addyosmani/agent-skills>

项目简介：这是 Google 工程师 Addy Osmani 推出的开源项目，把资深工程师在软件开发各阶段（定义、规划、构建、验证、审查、发布）使用的工程规范和工作流，封装成 AI 编程代理可调用的"技能包"。它解决的核心问题是：AI 代理写代码往往缺乏工程纪律，容易跳过测试、忽视质量门禁，这个项目让代理像资深工程师一样按流程办事。

核心功能： - **9 个斜杠命令：** /spec、/plan、/build、/test、/review、/ship 等，分别对应开发生命周期的不同阶段，每个命令会自动激活相关技能。 - **25 个技能包：**涵盖 API 设计、前端工程、TDD、代码审查、Web 性能审计等，可根据当前任务自动触发。 - **/build auto 自主模式：**批准一次计划后代理可自动执行所有任务，但每个任务仍坚持测试驱动、单独提交，遇到失败或风险步骤会暂停。 - **跨代理安装：**通过 `npx skills add` 一条命令即可装入 Claude Code、Cursor、Codex、Copilot 等 70+ 种代理。

适用场景：主要面向使用 AI 编程代理进行实际项目开发的工程师和团队，尤其是希望让 AI 代理产出"生产级"而非"玩具级"代码的场景。也适合个人开发者把资深工程习惯固化进日常 AI 协作流程中。

技术亮点：项目本质是"提示词工程 + 工作流编排"的产物，用 JavaScript 组织技能定义和命令映射。设计上强调"约束先行"（/constraints 一次定义、处处执行）和"测试即证明"的理念，同时通过自动技能触发机制，让代理无需人工选择就能应用恰当规范。它还兼容 Vercel 的 skills CLI 生态，具备良好的可移植性。

上手建议：最快的路径是运行 `npx skills add addyosmani/agent-skills --list` 浏览全部技能，再选择性安装。Claude Code 用户可直接通过插件市场安装。若想贡献，建议先阅读 skills/ 目录下的技能定义格式，注意 README 提到的单技能安装会缺失 references/ 共享清单这一已知问题（issue #361），贡献时可优先帮助完善这部分的可移植性。

#7

C

NEW

raddebugger

A native, user-mode, multi-process, graphical debugger.

Stars **7,768** Forks **380** Today **+82**

<https://github.com/EpicGames/raddebugger>

项目简介：RAD Debugger 是 Epic Games 开源的一款原生、用户态、多进程的图形化调试器，目前聚焦于 Windows x64 平台的本地调试，旨在为大型项目提供比传统工具链更高效的调试体验。它不只是一个调试器，还配套自研的 RDI 调试信息格式和 RAD 链接器，试图从底层重构整个调试工具链。

核心功能：一是原生多进程图形化调试，支持 Windows x64 + PDB 调试信息；二是自定义的 RAD Debug Info (RDI) 格式，可将 PDB/DWARF 按需转换，规避传统格式在大项目中的性能瓶颈；三是 RAD Linker，专为超大工程设计的快速链接器，兼容 MSVC 命令行，实测链接速度提升约 50%；四是 radbin 工具，支持调试信息格式转换与文本转储。

适用场景：主要面向需要调试超大型 C/C++ 项目的 Windows 开发者，尤其是那些因 PDB 体积膨胀、32 位表溢出或链接耗时过长而苦恼的团队。当前处于 ALPHA 阶段，更适合愿意尝鲜、反馈问题的早期用户，而非追求稳定性的生产环境。

技术亮点：RDI 格式直接绕开了 PDB 和 DWARF 的历史包袱，按需转换的设计既兼容现有生态又避免了长期依赖；RAD Linker 针对多 GB 级调试信息优化，还支持大内存页进一步提速 25%，并明确提示了大页在 Windows 下的内存碎片风险，工程判断相当务实。整个项目用 C 编写，代码即规范（RDI 格式定义直接写在 rdi.h/rdi.c 中），风格硬核。

上手建议：想使用的话，直接从 Releases 页面下载预编译二进制，README 中提到的调试器使用说明打包在发布版或本地 build 文件夹内。想贡献代码，先在 Windows x64 上装好 MSVC Build Tools 和 Windows SDK，按 README 的本地构建步骤编译；遇到问题积极提交 issue 并附上 dump 文件和复现步骤，对 ALPHA 阶段的项目帮助极大。

#8

TypeScript

连续 3 天

claude-mem

Persistent Context Across Sessions for Every Agent – Captures everything your agent does during sessions, compresses it with AI, and injects relevant context back into future sessions. Works with Claude Code, OpenClaw, Codex, Gemini, Hermes, Copilot, OpenCode + More

Stars **97,541** Forks **8,588** Today **+578**

<https://github.com/thedotmack/claude-mem>

AI 分析生成失败: Error code: 400 - {'error': {'message': 'Content Exists Risk (request_id: df755aaa-0afc-4209-a9a1-da19654a3c50)', 'type': 'invalid_request_error', 'param': None, 'code': 'invalid_request_error'}}

#9

Swift

NEW

cmux

Open source Ghostty-based macOS terminal with vertical tabs and notifications for AI coding agents. Built for multitasking, organization, and programmability.

Stars **27,760** Forks **2,458** Today **+96**

<https://github.com/manaflo-ai/cmux>

项目简介：cmux 是一款基于 Ghostty 构建的开源 macOS 终端，专为同时运行多个 AI 编程助手（如 Claude Code）的开发者设计。它解决的核心问题是：当你在多个 AI agent 之间切换时，如何一眼看出哪个任务需要你介入、哪个还在运行——传统终端对此几乎无能为力。

核心功能：

- **通知环与通知面板：**当 AI agent 需要关注时，对应面板亮起蓝环、标签高亮，侧边栏汇总所有待处理通知，一键跳转到最新未读。
- **垂直标签 + 分屏：**侧边栏显示 git 分支、关联 PR 状态、工作目录、监听端口和最新通知文本，支持横竖分屏。
- **内嵌浏览器：**可在终端旁分割出一个浏览器面板，并提供可脚本化的 API（移植自 agent-browser），方便调试 Web 应用。
- **SSH 工作区：**cmux ssh user@remote 直接为远程机器创建工作区，浏览器面板走远程网络，localhost 开箱即用。
- **Claude Code Teams 集成：**一条命令 cmux claude-teams 即可启动 Claude Code 的团队模式，队友以原生分屏形式出现，无需 tmux。

适用场景：主要面向同时驱动多个 AI 编程 agent 的开发者，尤其是需要并行处理多个任务、频繁在 agent 之间切换并等待人工确认的工作流。也适合喜欢 Ghostty 但想要更强多任务管理能力的终端用户，以及需要远程开发（SSH）并希望浏览器与终端联动的场景。

技术亮点：

- 原生 Swift + AppKit 构建，而非 Electron，启动快、内存占用低，同时通过 libghostty 实现 GPU 加速渲染。
- 完全兼容 Ghostty 配置（读取 ~/.config/ghostty/config），老用户迁移成本几乎为零。
- 提供 CLI 和 socket API，可编程地创建工作区、分屏、发送按键、自动化浏览器，具备较强的可扩展性。
- 浏览器面板支持从 Chrome、Firefox、Arc 等 20+ 浏览器导入 cookie 和会话，省去重复登录。

上手建议：直接从 releases 页面下载 macOS DMG 安装即可，已有 Ghostty 配置会自动生效。想深入定制可从 cmux.json 定义项目级自定义命令入手，再探索 CLI 和 socket API 做自动化。贡献方面，项目用 Swift 编写，建议先阅读官网的 "Zen of cmux" 博客理解设计理念，再从 issues 中找入门级任务切入。

#10

Rust

NEW

cua

Scale computer-use 2.0 with open-source drivers, cross-OS fleets, and benchmarks for training, evaluation, and data generation.

Stars **28,644** Forks **2,034** Today **+229**

<https://github.com/trycua/cua>

项目简介：Cua (trycua/cua) 是一个让 AI Agent 真正“使用电脑”的开源基础设施项目，目标是给智能体提供可操作的完整桌面环境。它解决的是当前 Agent 大多只能调用 API 或写代码、却难以操控图形界面软件的痛点，通过开源驱动、跨系统机群和评测基准，支撑“Computer-Use 2.0”场景。项目用 Rust 编写，已获得近 2.9 万星标，热度很高。

核心功能：一是 Cua Spaces，为 Agent 提供本地或自有云上的完整桌面（macOS/Linux 虚拟机），并支持“传送”已登录应用、人机同屏协作；二是 Cua Driver，跨 macOS/Windows/Linux 检查并操作应用；三是 Lume，在 Apple Silicon 上跑本地 macOS/Linux 虚拟机；四是 CUA-S1 决策小模型与 Cua Bench 评测基准，用于训练、评估和生成轨迹数据。

适用场景：适合做 GUI Agent 研究、训练和评测的团队，也适合需要让 AI 自动操作桌面软件（如浏览器、办公应用）的产品开发者。典型场景包括自动化测试、数据采集、Agent 基准测试，以及“人机协作”式的任务执行——人可随时接管再交还给 Agent。

技术亮点：用 Rust 实现底层驱动和虚拟机管理，强调跨操作系统与跨机器的规模化编排；Spaces 内置 cua-spacesd，让 Agent 启动即获得进程、文件、截图和输入能力。Keyvault 加密会话与“传送已登录应用”的设计，在安全性和可用性之间做了较细致的权衡，这是同类项目中较少见的工程化思路。

上手建议：普通用户可从安装 Cua Spaces 桌面应用开始（macOS 26+ 提供一键脚本），跑通第一个 Space；开发者建议先看 Cua Driver 和 Cua SDK/CLI 的快速开始，再按需深入 Lume 或 Cua Bench。想贡献代码，可优先关注 Rust 驱动层、跨平台适配和评测任务扩展，Discord 与官方文档是主要入口。

数据来源: github.com/trending

报告日期: 2026-10-07

由 DeepSeek AI 提供智能分析 | 自动生成